



SAM HIKEN

Garbage Collection Algorithms

What is garbage collection?

Founder of A Word.A.Day
(AWAD)

Automatic memory management

Advantages:

1. Lower risk of memory leaks and dangling pointers
2. More ergonomical

Why study garbage collection?

Founder of A.Word.A.Day
(AWAD)

Fact: *Most programming languages use garbage collectors*

Garbage collection comes with an **inherent trade-off**:

Aggressive garbage collection comes with a greater runtime

Contents

Founder of A.Word.A.Day
(AWAD)

1. Basic garbage collection algorithms [Wilson '94, Cheney '70, Collins '60]
2. Faster garbage collection [Baker '78, Appel '89, Boehm et al '91]
3. Conclusion

TEXT

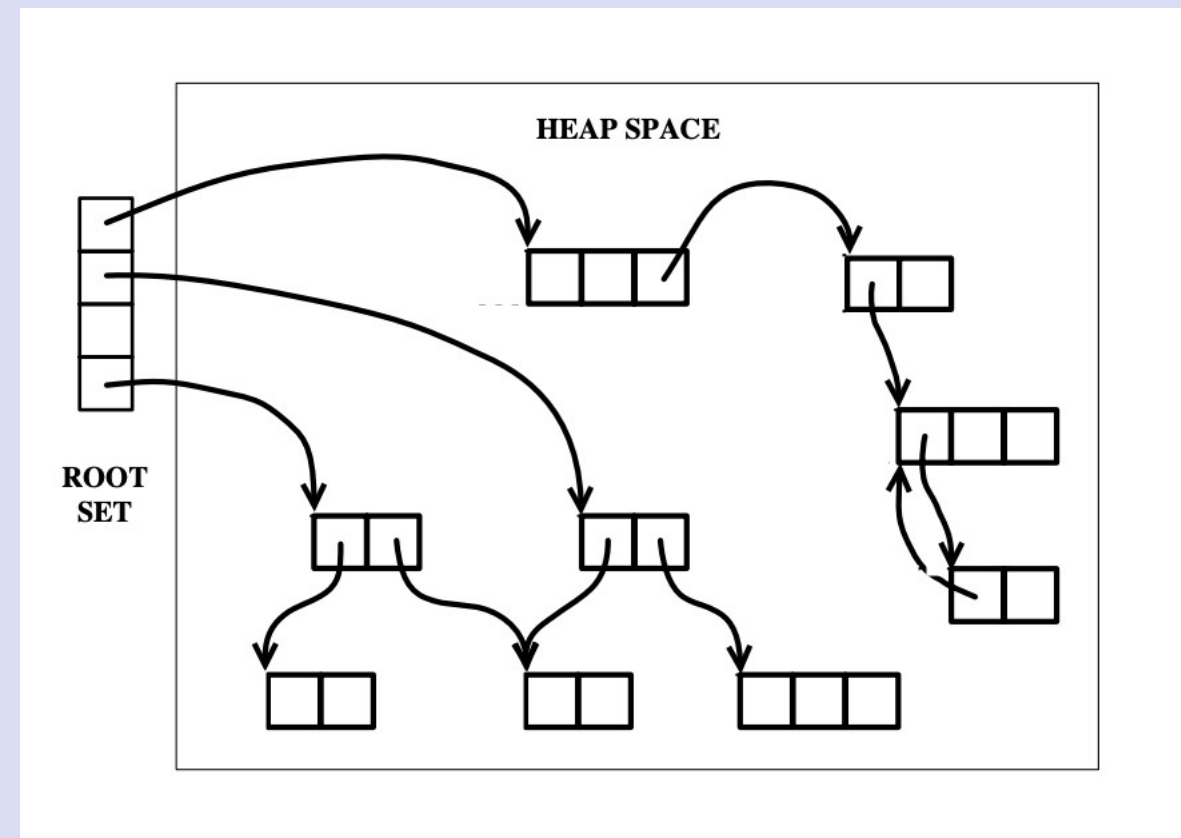
Founder of A.Word.A.Day
(AWAD)

Part 1: Basic garbage collection

Terminology

Founder of A.Word.A.Day
(AWAD)

1. **Mutator** (client program): Maintains an **object graph**:
 1. Nodes = objects in the heap
 2. Edges = $u \rightarrow v$ exists if u has a reference to v
 3. Certain nodes are designated **mutator roots**
2. **Collector**: algorithm that frees memory not reachable from the mutator roots



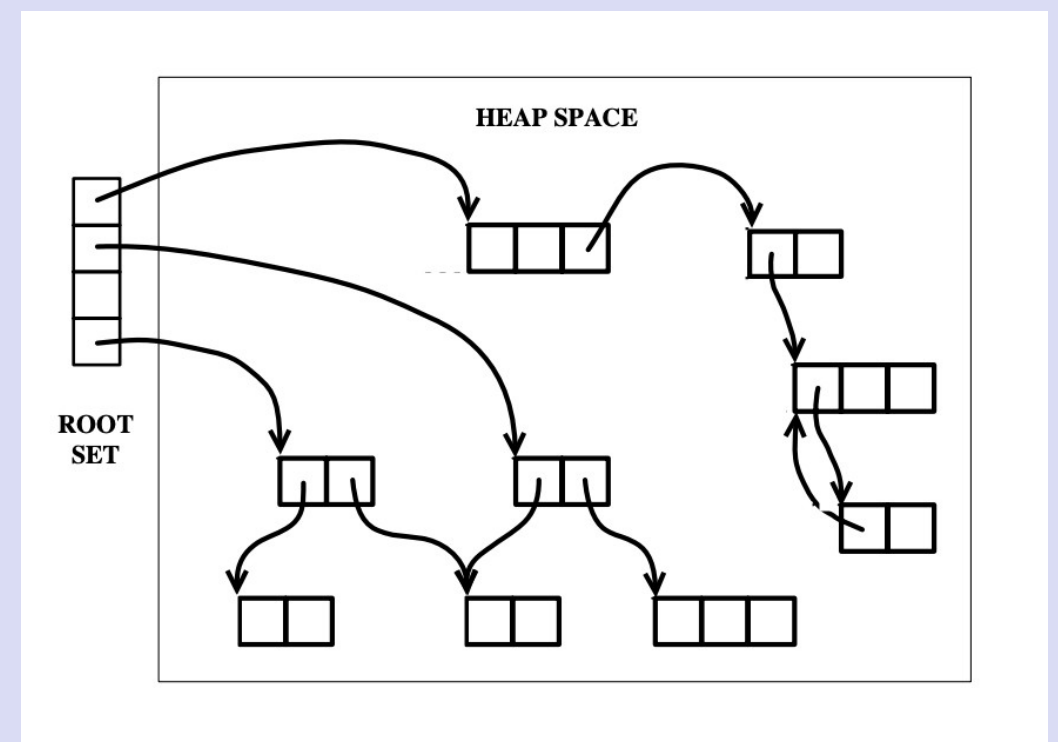
Algorithm 1: mark-sweep

Founder of A.Word.A.Day
(AWAD)

(see e.g. [Wilson '94])

Periodically does the following:

1. **Mark phase:** Run tree traversal from the mutator roots, “mark” all memory locations reached
2. **Sweep phase:** scan through heap, reclaim unmarked locations

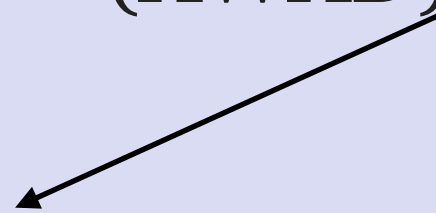


Algorithm 1: mark-sweep

Founder of A.Word.A.Day
(AWAD) see “mark-compact” algorithm

Disadvantages:

1. Live objects become interspersed with reclaimed memory
2. Sweep phase scans *the entire heap*



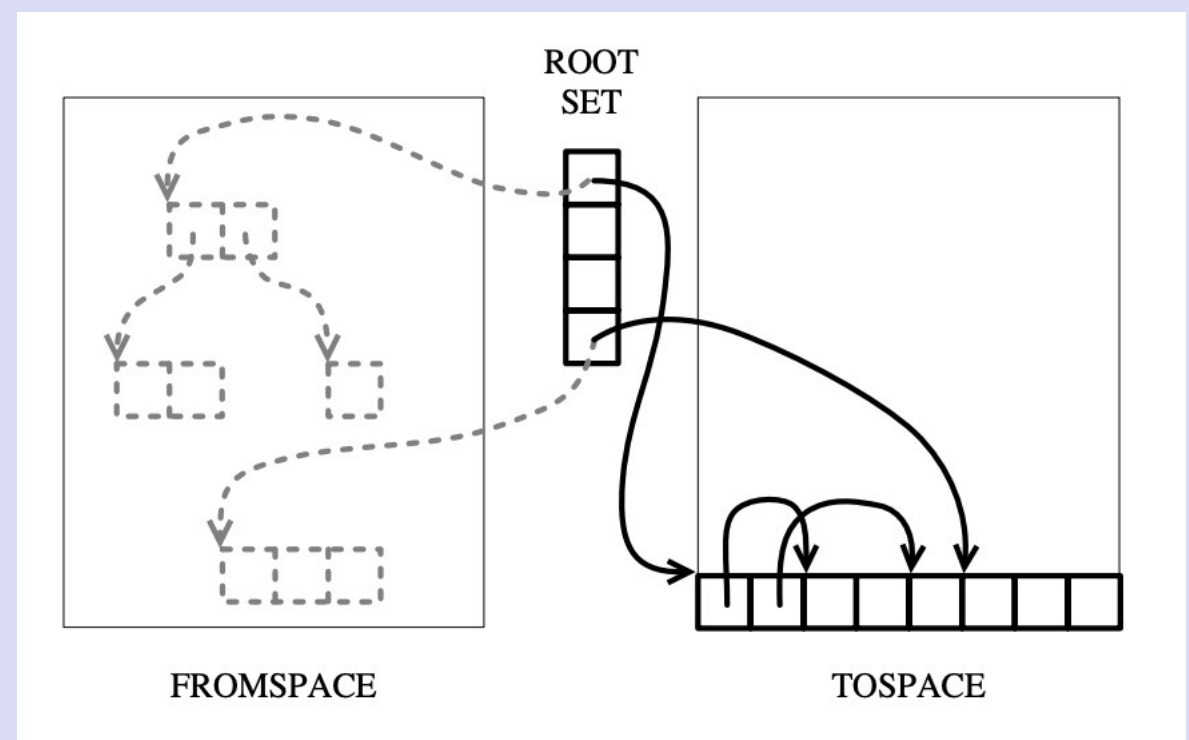
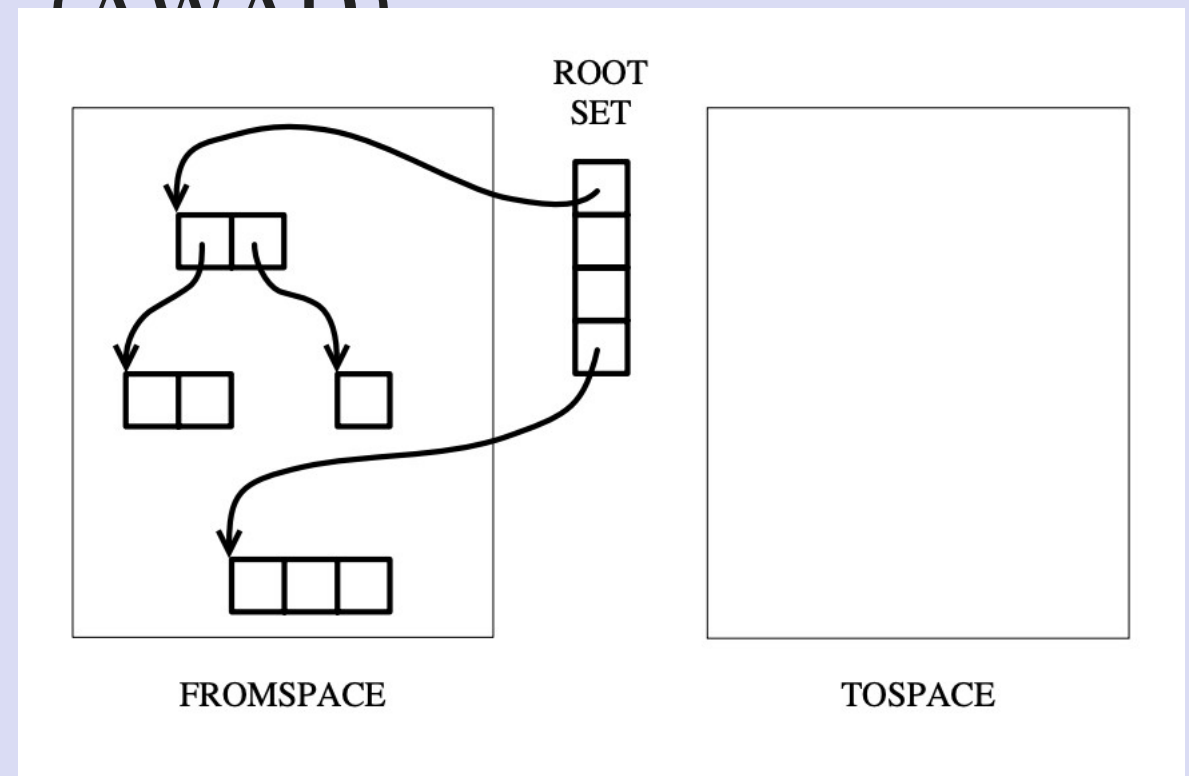
Algorithm 2: copying garbage collection

Founder of A.Word.A.Day
(AWAD)

Cheney's stop-and-copy algorithm

[Cheney '70]:

1. Maintain two “semi-spaces” in the heap: *to-space* and *from-space*
2. When objects are created, memory is allocated in the from-space
3. When the from-space becomes nearly full, *copy* live objects to the to-space
4. Designate the to-space the current from-space, and vice-versa



Algorithm 2: copying garbage collection

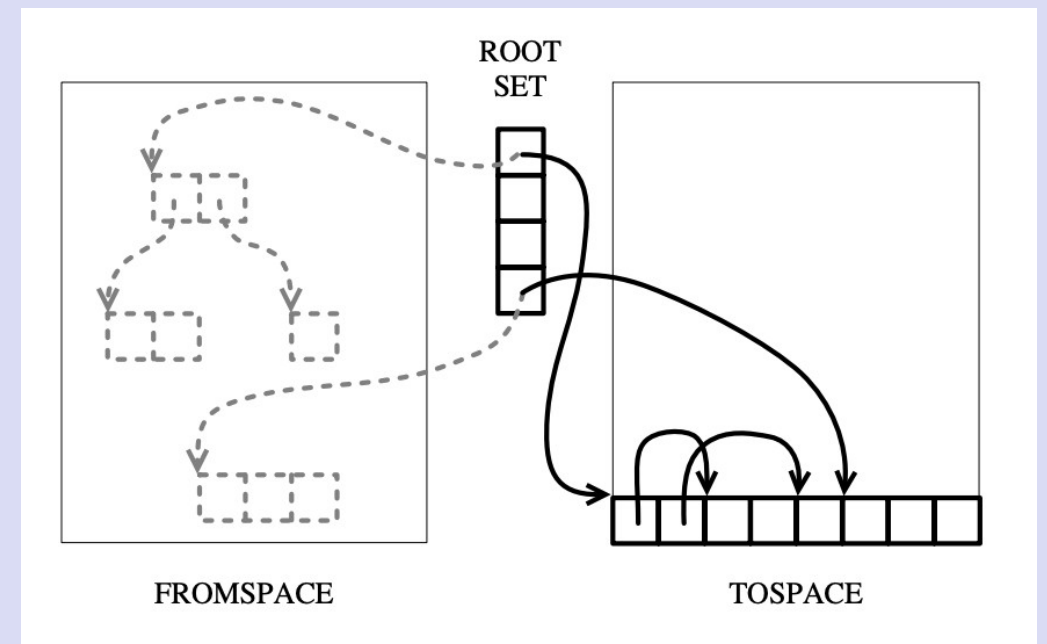
Founder of A.Word.A.Day
(AWAD)

Advantages:

1. Items are copied over into contiguous spots in memory.
2. Runtime is proportional to the number of *live objects* in the heap, rather than to the entire heap

Disadvantage:

3. Worst-case update time is still not fast enough for many applications, e.g. real-time computing



Algorithm 3: reference counting

Founder of A Word A Day
(AWAD)

So far, the algorithms we have seen have been *tracing algorithms*.

I.e., they traverse through the object graph

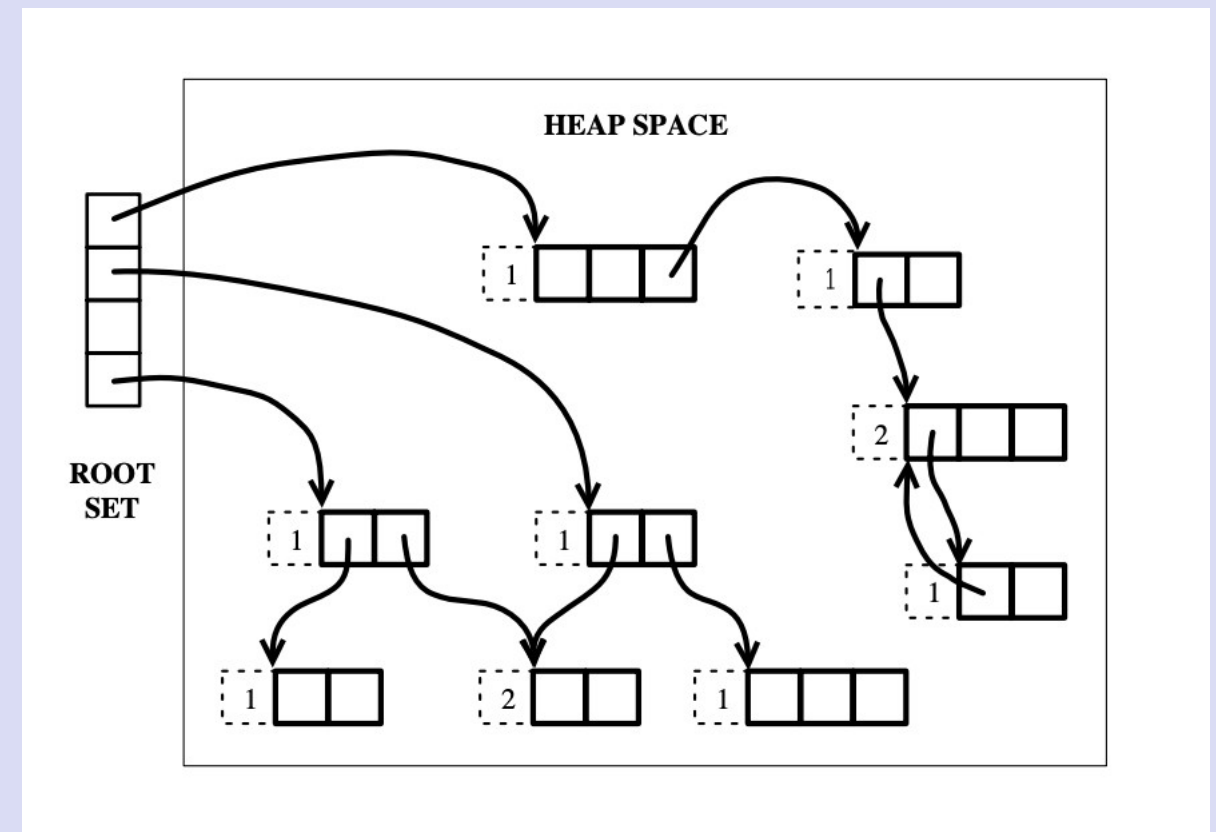
Such is not the case with **reference counting**, introduced by [Collins '60]



Algorithm 3: reference counting

Founder of A Word A Day
(AWAD)

1. Each object has an associated “reference count,” which tracks how many references to it exist.
 1. When a reference to a is created, a ’s count is incremented
 2. If b references a , then when b ’s memory is reclaimed, a ’s count is decremented
2. When an object’s count goes from 1 to 0, the memory it occupies is reclaimed.



Algorithm 3: reference counting

Founder of A-Word-A-Day
(AWAD)

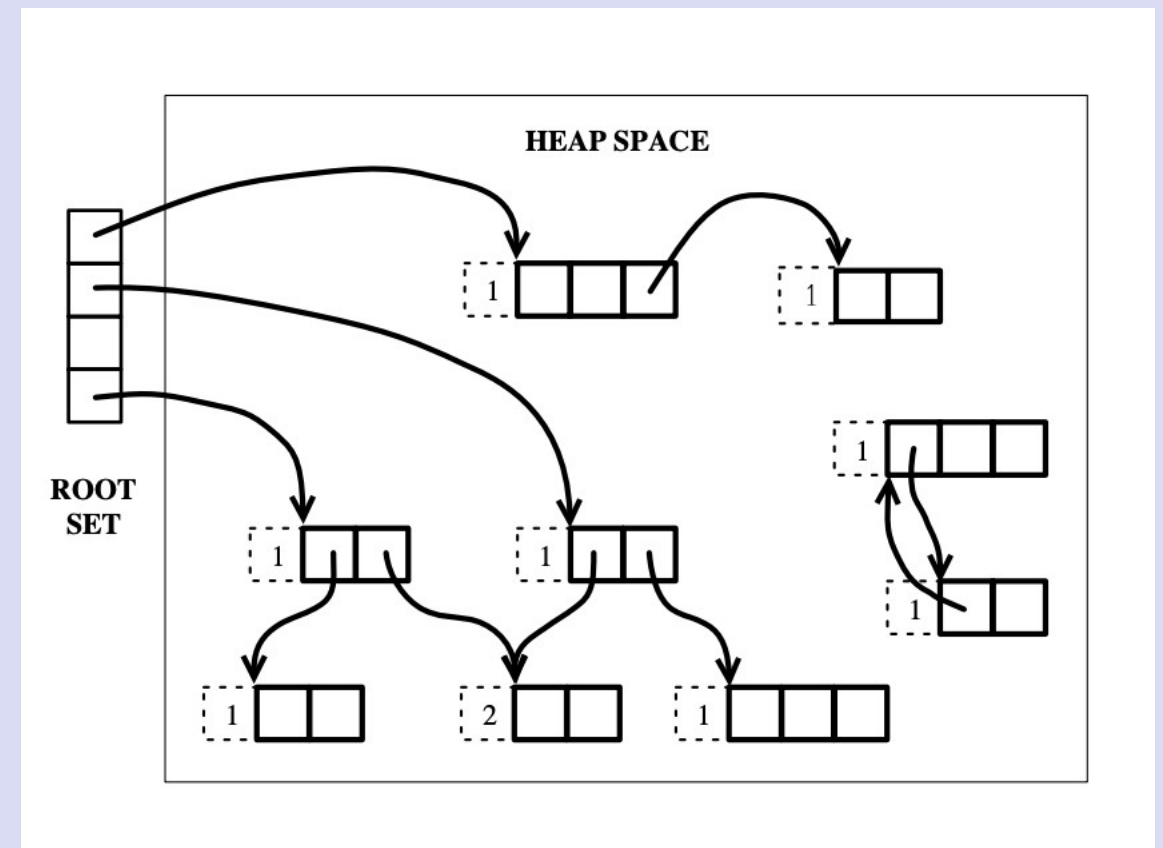
Advantage: it's *incremental*; i.e.,
memory is reclaimed immediately

Prevents very expensive heap scans

Makes it possible to use with real-
time systems

Disadvantage: fails to reclaim cycles

Can we design an incremental tracing
algorithm?



TEXT

Founder of A.Word.A.Day
(AWAD)

Part 2: Faster garbage collection

Incremental tracing garbage collection

Founder of A.Word.A.Day
(AWAD)

Goal: tracing garbage collection that updates the heap incrementally over time

Enter: **Baker's incremental copying scheme** [Baker '78]

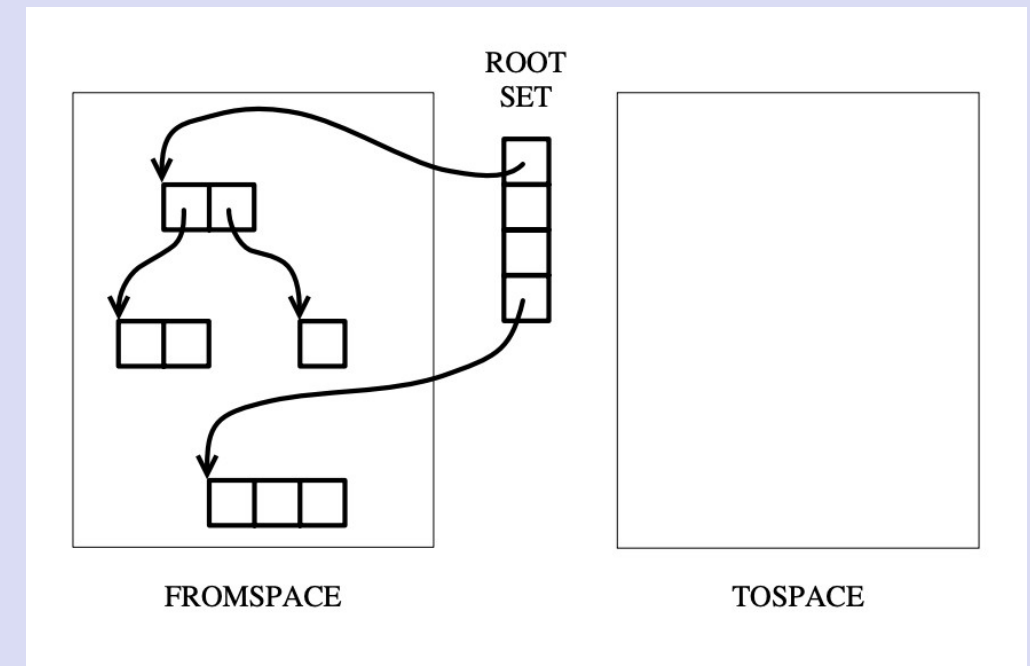


Incremental tracing garbage collection

Founder of A.Word.A.Day
(AWAD)

Algorithm: Copying collection, done incrementally

1. Designate from-space and to-space
2. When a collection cycle begins, all objects incident to root nodes are moved from from-space to to-space
3. Program execution resumes
 1. Whenever an object located in the from-space is accessed, it is moved to to-space
4. Eventually, from-space and to-space are flipped; next cycle begins



Incremental tracing garbage collection

Founder of A.Word.A.Day
(AWAD)

Advantages:

1. Avoids really expensive updates happening all at once (like reference-counting)
2. Detects cycles (like tracing algorithms)

But, we can make collection even faster

Generational garbage collection

Founder of A World A Day
(AWAD)

Generational Hypothesis: “most objects die young”

I.e, most objects are used for a short period of time before going out of scope

Garbage collectors should prioritize reclaiming memory that has been recently allocated

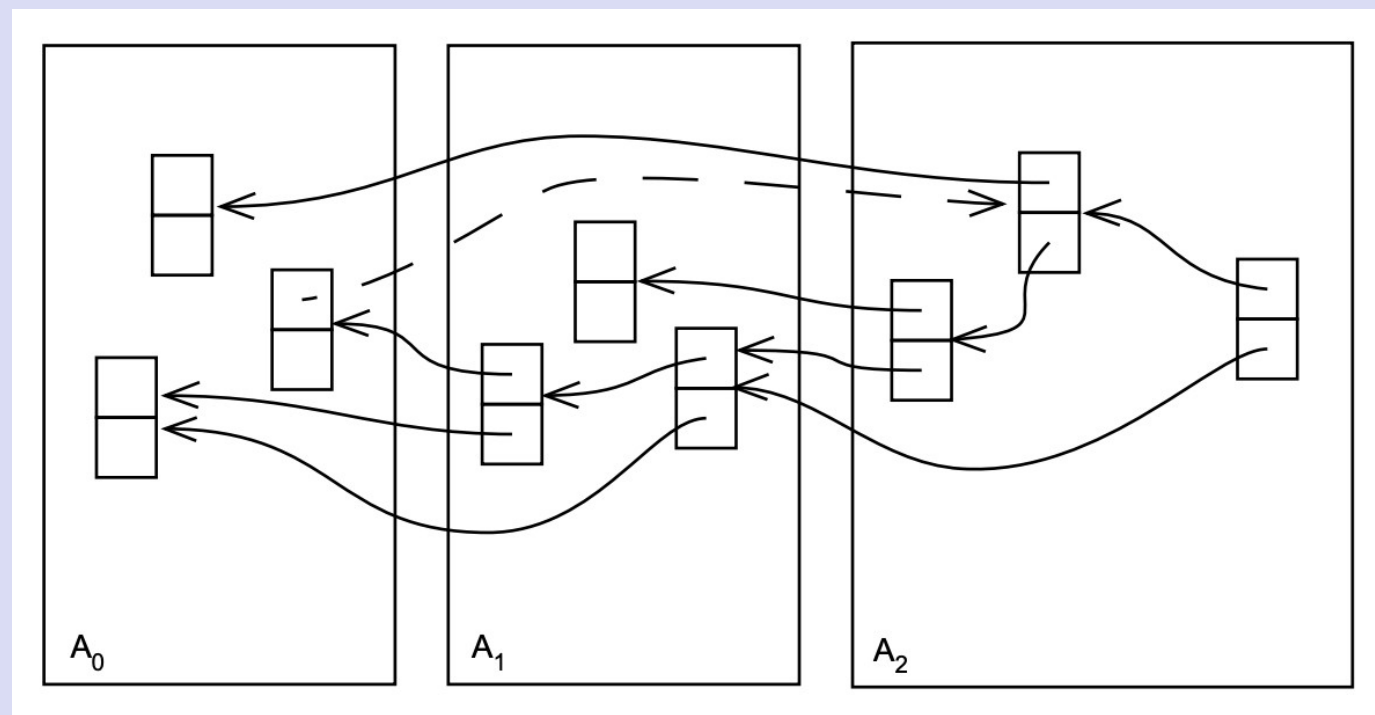
Observation: in most object graphs, most edges point from newer objects to older objects

Generational garbage collection

Founder of A. Word. A. Day
(AWAD)

Example algorithm: [Appel '89]

1. Idea: when running tree traversal, stop exploring once you reach nodes that are “sufficiently old”
2. Because most edges go from newer to older objects, the traversal has explored most objects that are fairly new

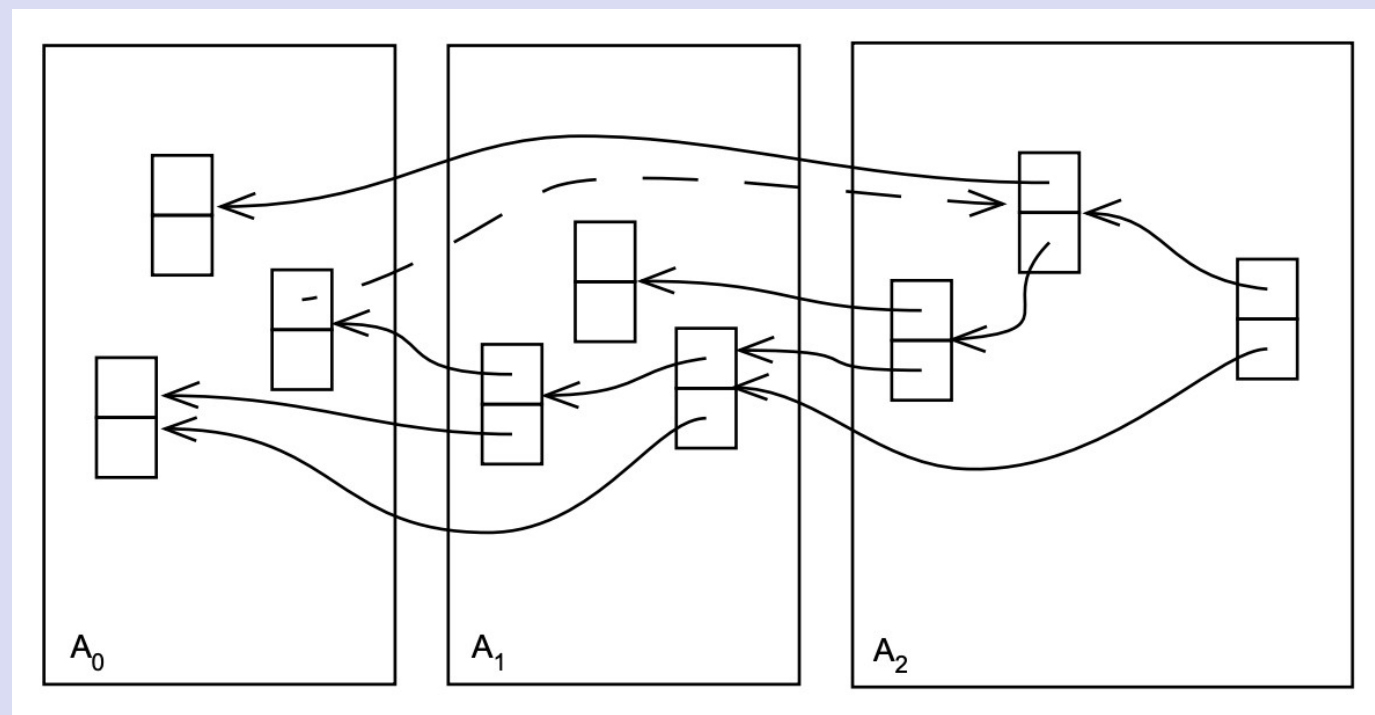


Generational garbage collection

Founder of A. Word. A. Day
(AWAD)

Implementation:

1. Divide heap into some number of areas, where objects in the same area are roughly the same age
2. Run a copying collector that is (mostly) restricted to exploring nodes in areas containing more recently created objects
3. Use more sophisticated techniques to handle edges going from older to younger objects



Generational garbage collection

Founder of A. Word.A. Day
(AWAD)

Big picture:

1. Pause times are substantially lower than non-generational algorithms
2. Most garbage collectors implemented today use generational garbage collection

Can we make the pause times even shorter?

Parallel garbage collection

Founder of A.Word.A.Day
(AWAD)

[Boehm et al. '91] “**Mostly Parallel Garbage Collection.**”

In the algorithms we’ve seen so far, the execution of the program inevitably needs to be paused while the collector is run.

Idea: Run the collector and client program *in parallel*

Boehm et al. '91 — Overview

Founder of A. Word.A. Day
(AWAD)

Most garbage collectors work in a “**stop-the-world**” fashion

I.e, when the collector is running, the program itself is paused

Boehm et al. offer a **general technique** for converting stop-the-world tracing garbage collectors into *mostly parallel* collectors

Boehm et al. '91 — Basic idea

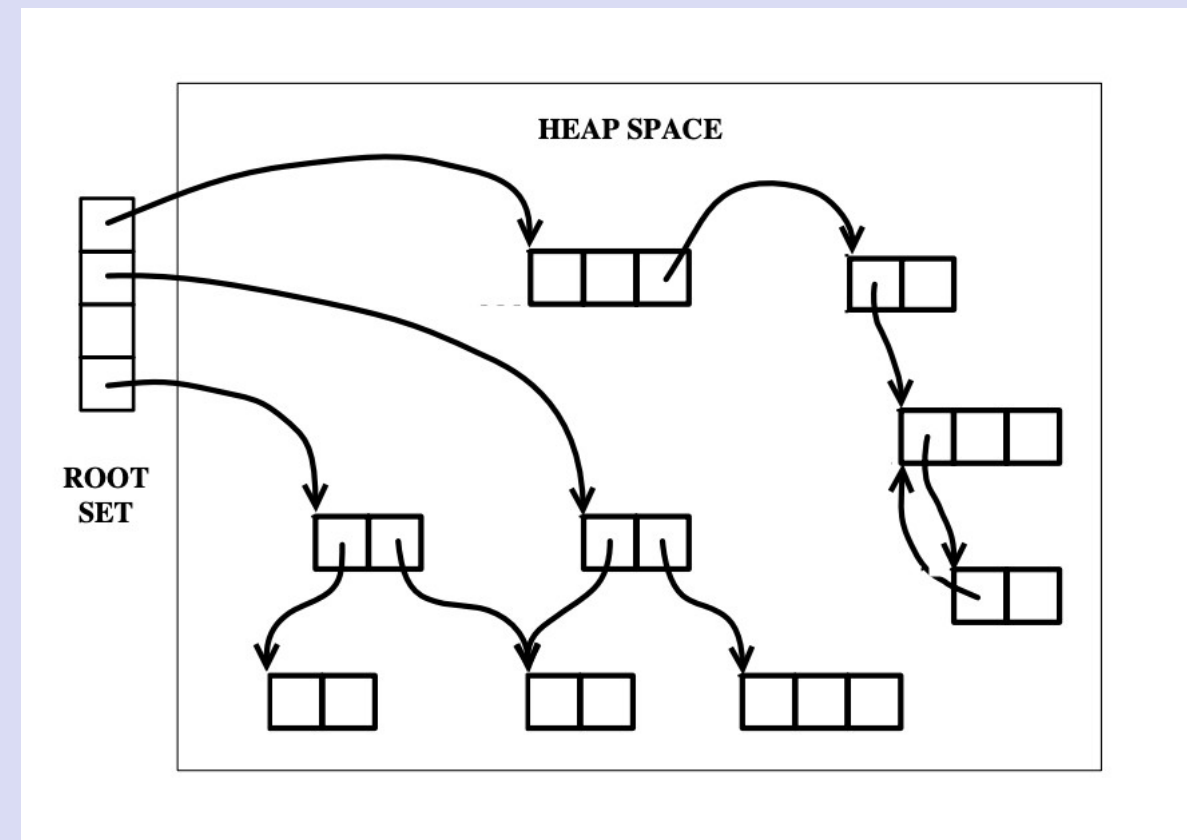
Founder of A. Word.A.Day
(AWAD)

Let us see how this technique works for the naive mark-sweep garbage collector.

Recall the mark-sweep algorithm:

Periodically do the following:

1. **Mark phase:** Run tree traversal from the mutator roots, “mark” all memory locations reached
2. **Sweep phase:** scan through heap, reclaim unmarked locations



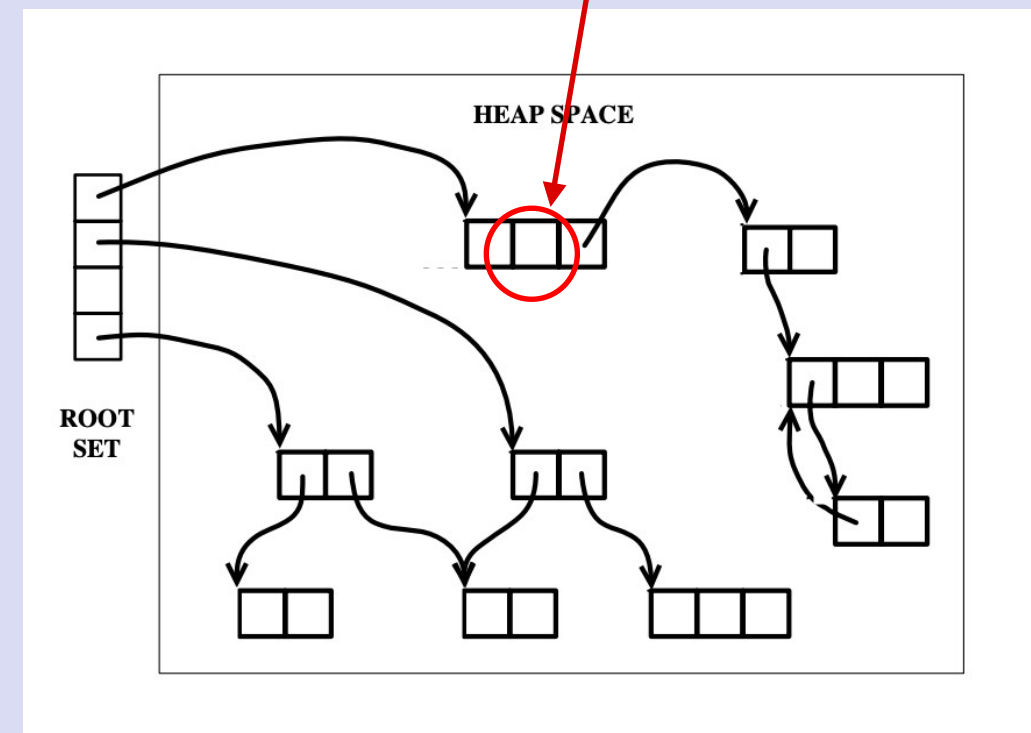
Boehm et al. '91 — Basic idea

Founder of A. Word.A. Day
(AWAD)

Mark phase:

1. Maintain some “virtual memory” outside of the heap
2. Periodically do the following:
 1. Run tree traversal on the object graph *while still running the program*
 2. Meanwhile, whenever the program modifies a location in the heap, store a pointer to that location in the virtual memory
 3. When the traversal is finished, stop-the-world and run tree traversal from the pointers in the virtual memory

Virtual
memory



Boehm et al. '91 — Basic idea

Founder of A. Word.A. Day
(AWAD)

Sweep phase:

1. Idea: interleave sweeping with future memory allocation
2. Divide the heap into blocks, where each block contains objects of roughly the same size
3. Whenever memory for a new object needs to be allocated, sweep through the corresponding block until an unmarked location is found

Big picture: the runtime is dominated by the mark phase

Boehm et al. '91 — Theoretical analysis

Founder of A.Word.A.Day
(AWAD)

The pause time is dominated by the mark phase.

Recall how the mark phase is performed:

1. Run tree traversal on the object graph *while still running the program*
2. Meanwhile, whenever the program modifies a location in the heap, store a pointer to that location in the **virtual memory**
3. When the traversal is finished, stop-the-world and **run tree traversal from the pointers in the virtual memory**

The **pause time** is proportional to the number of **pointers in virtual memory**

The number of **pointers in virtual memory** is proportional to the number of **memory locations modified** during the parallel operation.

This can get quite expensive during periods where the client program is very active

Boehm et al. '91 — Empirical tests

Founder of A.Word.A.Day
(AWAD)

The authors implemented a mostly parallel, generational garbage collector

Compared to:

1. a non-parallel non-generational algorithm
2. a non-parallel generational collection algorithm

Used two benchmarks:

3. “Boyer” benchmark — widely used benchmark in garbage collection
4. *Allocator loop* in C — program that loops and allocates memory

Boehm et al. '91 — Empirical results

Founder of A.Word.A.Day (AWAD)

“full” = non-generational, non-parallel collector

“gen” = generational, non-parallel

“gen, par” = generational, parallel

10 MB, alloc20meg

	Total time <i>secs</i>	No. of colls. (full)
full	332.2	3(3)
gen	24.4	20(0)
gen,par	32.0	11(0)

Pause Times	
Max.	Ave.
<i>msecs</i>	<i>msecs</i>
51350	46323
870	125
350	102

10 MB, Boyer

	Total time	No. of coll.
full	512.6	4(4)
gen	360.6	22(5)
gen,par	259.6	12(2)

Max.	Ave.
63380	56548
41840	9291
329	169

Boehm et al. '91 — More empirical results

Founder of A. Word.A. Day
(AWAD)

“full” = non-generational, non-parallel collector

“gen” = generational, non-parallel

“gen, par” = generational, parallel

36 MB, alloc20meg

	Total time	No. of coll.	Max.	Ave.
full	16.4	3(3)	1040	1037
gen	15.5	17(0)	200	81
gen,par	17.2	11(0)	100	76

36 MB, Boyer

	Total time	No. of coll.	Max.	Ave.
full	51.6	4(4)	1610	1368
gen	60.4	22(5)	1471	528
gen,par	66.1	13(2)	159	134

Boehm et al. '91 — Future directions

Founder of A.Word.A.Day
(AWAD)

Open questions (in the '90s):

1. The parallelization technique described does not work for copying collection
 1. The paper proposes a technique for parallel copy collection, but does not implement it
2. Further reducing the amount of tracing that occurs during “stop-the-world” period

In the years since, parallel and concurrent garbage collectors have become the norm

TEXT

Founder of A.Word.A.Day
(AWAD)

Conclusion

Summary:

Founder of A.Word.A.Day
(AWAD)

1. Basic garbage collection algorithms

Tracing vs reference counting

2. Faster garbage collection

Incremental tracing collection

Generational collection

Parallel collection

Open problems

Founder of A.Word.A.Day
(AWAD)

1. Designing garbage collection for programs that use massive amounts of memory
2. Further improvements to concurrent garbage collection
 1. Many garbage collectors suffer when the heap becomes close to full
3. Adaptive tuning?



SAM HIEN

Questions?

Time vs. space complexity of Cheney's algorithm

Founder of A.Word.A.Day
(AWAD)

[maybe combine this with earlier slide / somehow condense it]

Cheney's algorithm allows for a proportional trade-off between the time complexity (the number of times each object is visited) and the space complexity (the total memory taken up)

Idea: set the size of the semi-spaces

1. Larger semi-spaces mean the average object is copied fewer times
2. Smaller semi-spaces mean less memory is used

Copy subroutine in Cheney's algorithm

Founder of A.Word.A.Day
(AWAD)

[maybe cut this slide]

Algorithm:

1. Maintain a queue Q of memory locations in the from-space
2. Initialize Q with the mutator roots
3. Increment through Q :
 1. For each unvisited object a referenced by $Q[0]$:
 1. Mark a as visited, then add a to end of Q
 2. Allocate space for $Q[0]$ in the to-space
 3. Reclaim memory allocated for $Q[0]$ in the from-space
 4. Update pointers to $Q[0]$ in the to-space
 5. Remove $Q[0]$ from Q

Algorithm 2: mark-compact

Founder of A.Word.A.Day
(AWAD)

Algorithm: periodically does the following:

1. *Mark phase:* run tree traversal from mutator roots
2. *Compact phase:* scan through memory;
 1. Whenever a live object is encountered, “slide” it to be adjacent to the previous live object

Advantage: avoids fragmentation issues of mark-sweep

Disadvantage: more computationally expensive

Mutators and collectors

Founder of A.Word.A.Day
(AWAD)

Garbage collection programs are split into two parts:

1. *Mutator*: maintains an *object graph*, a directed graph where
 1. Nodes = objects in the heap
 2. Edges = $u \rightarrow v$ exists if u has a reference to v
 3. *Mutator roots*: e.g, references in the call stack
2. *Collector*: discovers unreachable objects and reclaims the memory